```
$ ruby -v
ruby 3.2.0 (2022-12-25 revision a528908271) [arm64-darwin22]
```

This illustrates an important point. Most of the code samples you see in this book are actually executed each time we format the book. When you see some output from a program, that output was produced by running the code and inserting the results into the book.

Notation Conventions

Literal code examples are shown using a sans-serif font:

```
class SampleCode
  def run
    #...
  end
end
```

Within the text, Fred#do_something is a reference to an instance method (in this case, the method do_something) of class Fred, new. Class methods are marked as Fred.new, and Fred.EOF is a class constant. In some other Ruby documentation, you may see class methods written as Fred::new. This is perfectly valid Ruby syntax; we just happen to think that Fred.new is less distracting to read.

The decision to use a hash character to indicate instance methods was a tough one. It isn't valid Ruby syntax, but we thought that it was important to differentiate between the instance and class methods of a particular class. When you see us use File.read, you know we're talking about the class method read. When instead we use File#read, we're referring to the instance method read. This convention is standard in most Ruby discussions and documentation.

This book contains many snippets of Ruby code. Where possible, we've tried to show what happens when they run. In simple cases, we show the value of expressions on the same line as the expression. Here's an example:

```
a = 1
b = 2
a + b     # => 3
```

Here, you can see that the result of evaluating a + b is the value 3, shown in a comment at the end of the line, # => 3. If you simply typed this fragment of code into a file and executed it using Ruby, you wouldn't see the value 3 output—you'd need to use a method such as puts to have the values written to program output.

```
a = 1     # => 1
a + 2     # => 3
```

If the program produces more complex output, we show it after the program code:

```
3.times { puts "Hello!" }
```

produces:

```
Hello!
Hello!
Hello!
```

In some of the library documentation, we wanted to show where spaces appear in the output. You'll see these spaces as ␣ characters.